



Protocol Audit Report

Version 1.0

0xscarfac3

April 7, 2026

Protocol Audit Report

0xscarfac3

April 7, 2026

Prepared by: 0xscarfac3 Lead Auditors: - 0xscarfac3

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Storing the password on chain makes it visible to anyone, and no longer private
 - * [H-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec indicated a parameter that doesn't exist, Causing the natspec to be incorrect

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and it is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The 0xscarfac3 team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The Findings described in this document correspond the following commit hash:

```
1 7d55682ddc4301a7b13ae9413095feffd9924566
```

Scope

```
1 ./src/  
2 #--PasswordStore.sol
```

Roles

-owner : The user who can set the password and read the password. -outsiders : No one else should be able to read and set the password.

Executive Summary

Add some notes about how audit went, types of things u found, etc.

We spent X hours on Y audit using Z tools, etc.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	2

Findings

High

[H-1] Storing the password on chain makes it visible to anyone, and no longer private

Description: All data stored on-chain is visible to everyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be private and only accessed through the `PasswordStore::getPassword` function which is intended to be only called by the owner of the contract.

We show one such method of reading any data off chain below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: (Proof of code)

The below test case shows how anyone can read the password directly from the blockchain

- ### 1. Create a locally running chain

```
1 make anvil
```

- ## 2. Deploy the contract on the chain

```
1 make deploy
```

- ### 3. Run the storage tool

we use 1 because that's the storage slot of `s_password` in the contract.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url 127.0.0.1:8545
```

You will get the output looks like this :

[illegible]

Now, you can parse the hex string with :

```
cast parse-bytes32-string 0x6d79506173737766726400000000000000000000000000000000000000000000
```

And get an output of

```
1 myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

[H-2]PasswordStore::setPassword has no access controls, meaning a non-owner could change the password

Description: The `PasswordStore : : setPassword` function is set to be an `external` function. However, the natspec of the function overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
1 function setPassword(string memory newPassword) external {
2     //@audit = There are no access controls
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

Impact: Anyone can change/set the password of the contract, severely breaking the contract intended functionality.

Proof of Concept: Add the following code to `PasswordStore.t.sol`:

Code

```
1 function test_anyone_can_set_password(address randomAddress) public
2 {
3     vm.assume(randomAddress != owner);
4     vm.prank(randomAddress);
5     string memory expectedPassword = 'myNewPassword';
6     passwordStore.setPassword(expectedPassword);
7
8     vm.prank(owner);
9     string memory actualPassword = passwordStore.getPassword();
10    assertEq(actualPassword, expectedPassword);
11 }
```

run the following command on terminal:

```
1 forge test --mt test_anyone_can_set_password
```

Now u can make sure tha

Recommended Mitigation: Add an access control to the `setPassword` function.

Code

```
1 if(msg.sender != s_owner){
2     revert PasswordStore__NotOwner();
3 }
```

Informational

[I-1] The PasswordStore::getPassword natspec indicated a parameter that doesn't exist, Causing the natspec to be incorrect

Description

```
1      /*
2      * @notice This allows only the owner to retrieve the password.
3      * @param newPassword The new password to set.
4      */
5      function getPassword() external view returns (string memory) {
```

The natspec for the function `PasswordStore::getPassword` indicates it should have a parameter with the signature `getPassword(string)`. However, the actual function signature `getPassword()`.

Impact: The natspec is incorrect

Recommended Mitigation Remove the incorrect natspec line.

```
1      * @param newPassword The new password to set
```